# Queens, Donuts and Genetic Algorithms: a Guide

## Introduction

This report explores the application of a Genetic Algorithm (GA) to solve the eight queen problem and its extension to a toroidal board, comparing its application to another nature based optimisation algorithm – Particle Swarm Optimisation (PSO). Part 1 details the features of the GA implementation and the reasons behind them, providing analysis behind the implementation of the main distinguishing feature of this implementation: extinction events. Part 2 explains the challenge of a toroidal board and contextualises its implementation in the accompanying Jupyter notebook. Part 2 concludes with a comparison of GAs with PSOs and the potential for a practical application of the techniques seen in this assignment.

## Part 1: Genetic Algorithm for Classic 8-queen problem

### A. Initial architecture for the Genetic Algorithm

Aside from initialising with a random population as per assignment parameters, the choice of features for the GA followed the principles of parsimony and precedence in literature. Design choices also responded to the analysis showing the issue of stagnation in a local optimum, explored in part B. The initial GA had the following features:

- *Fitness:* Existing implementations of the 8-queen problem use the number of non-attacking pairs of queens (Tsou 2021, and Educative, n.d.). This approach is simple, has a natural interpretation, and is easily implemented by subtracting the cost function from the EightQueensState from the total number of pairs of queens.
- *Selection:* roulette, rank-based, and tournament selection methods are featured in other implementations to this problem (Educative, n.d.) and tournament selection was found to be the most effective with cursory experimentation, as well as being popular in a wide range of GA applications (Fang and Li 2010).
- *Crossover*: the chosen single-point crossover function is one of the simplest to implement and quickest to run. Although Hasançebi and Erbatur (2024) find more complex crossover functions to be optimal for convergence to solution in their test problems, they note that "in the high values of generation number, the other [than single] crossover techniques are subjected to a significant reduction in their exploration capability." (pp. 441-2). As shown in part B, insufficient exploration at higher generations was the primary challenge to the working of the GA.
- *Mutation:* By randomly changing the position of a random queen with a given probability within each child, an element of randomness is introduced to encourage exploration of alternative solutions, at the cost of more rapid convergence to a solution. My implementation introduces a dynamic mutation rate that starts high to encourage exploration in early generations and drifts lower to enable exploitation in later generations. It also raises the mutation rate temporarily if there is no improvement in the fitness of the best individual for a given number of generations, in order to dislodge the algorithm from a local optimum.

- *Elitism:* GAs featuring elitism – the carrying over of the best individuals from each generation to the next generation – converge to solutions more rapidly, and have become best practice for GAs (Greenstein, Elsey, and Hutchinson 2023). Moreover, the introduction of elitism in my GA served as a way to help modulate the other design choices that sacrificed exploration for exploitation: the single-point crossover and dynamic mutation rate.

**B. Analysis of first pass results and introduction of extinction feature**

Figure 1: GA can gets stuck in local optimum

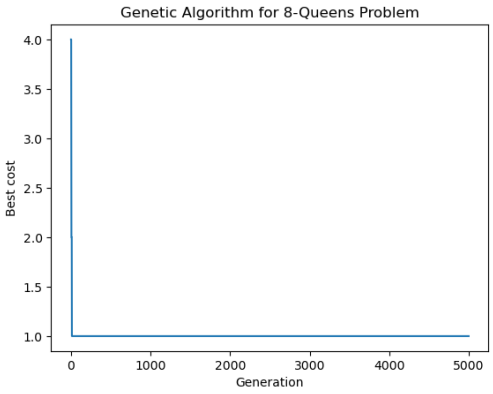
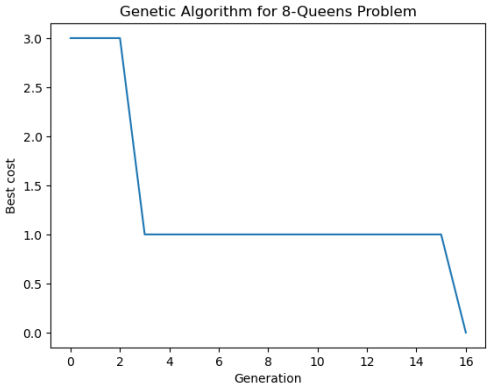



Figure 2: GA can converge quickly to a solution




**Note:** The y axis displays the cost of the best individual in each generation rather than the fitness – i.e. the number of attacking queens

Despite exploration-focused architecture, the GA would often get stuck in a local optimum from which it would not dislodge even after thousands of generations (figure 1) and take well over 30 second to run. In other instances it could converge rapidly to a correct solution (figure 2). The local optimum problem persisted over a wide range of parameter combinations, suggesting a solution lay in the GA architecture rather than in parameter optimisation.

The importance of starting conditions in determining the success of the algorithm led me to introduce an "extinction" feature, whereby the population resets if the fitness of the best individual does not increase for a set number of generations. This approach has precedence in the literature (Jaworski et al 2012), although I conceived of it originally, before reviewing other approaches. Keeping the elites from the pre-extinction generations was tried, but kept the algorithm in a local optimum trap, leading to a complete reinitialization of the population in each extinction event. This solution led to the consistent solving of the 8-queen problem in under 30 seconds, as demonstrated in a histogram of completion times in the submitted Jupyter notebook

## Part 2: Extension of the problem to a toroidal board and comparison to Particle Swarm Optimisation

Taking a classic chess board, wrapping it into a cylinder, and joining the ends to produce a donut shape gives a toroidal board. Though a queen's reach along rows and columns remains the same, it is extended along the diagonals, presenting a more challenging

problem for the GA (see figure 3 for an illustration). The implementation of this problem was made by altering the cost function of the EightQueensState class and facilitating the GA to work for multiple board sizes. This latter alteration was necessary since the toroidal queens problem cannot be solved for boards with an n by n size where n is a multiple of two or three (Cook 2021). Figure 4 illustrates the results of setting the GA on an 8-queen toroidal problem. Even after 10,000 total generations, with multiple extinction events resetting the population, the GA fails to find a (non-existent) solution. Conversely, the Jupyter notebook submission demonstrates that solutions are found for boards of size 7 by 7 and 11 by 11, which are not multiples of two or three, demonstrating sound programming.

Figure 3: Illustration of extend reach on toroidal board

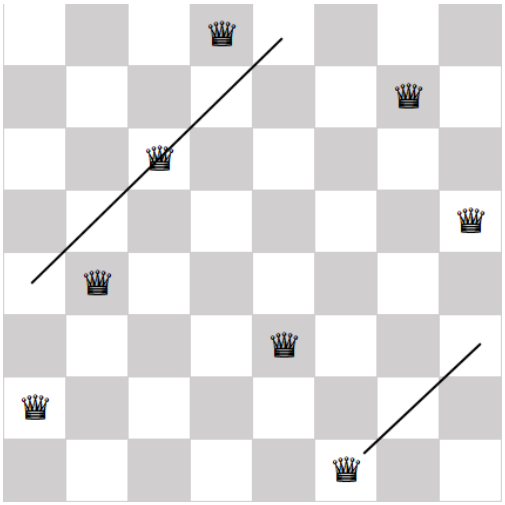
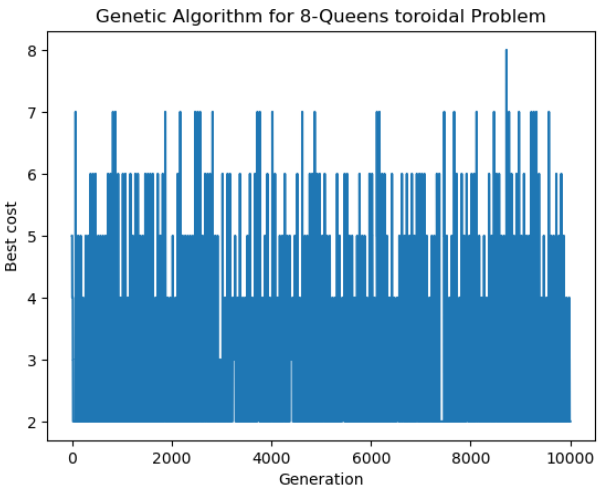


Figure 4: Illustration of insolvability of 8-queen toroidal board




**source**: Cook 2021

PSO shares many similarities with GAs in that they both initialise populations of solutions randomly and proceed in an iterative process that is directed by individuals' fitness. The algorithms differ in how they traverse the solution space, with PSOs relying on acceleration of individuals towards their and their neighbour's previous optimal values and GAs relying on reproduction. A comparison of GAs and PSOs on the eight-queen problem suggest that whilst they perform similarly for the classic 8-queen problem, PSOs take longer to solve scaled-up versions of the problem (Hammand and Thannoon 2007), and therefore would likely take longer to solve the more complex toroidal board problem, making a GA the preferred choice. The local optimum problem may manifest more gravely in PSOs without as many mechanisms as GAs to encourage exploration (such as mutation). However, an analogue to extinction events could improve PSO performance as it has done with the GA. The highly explorative GA with an extinction feature, or another nature-based algorithm with an analogous feature, could be gainfully employed for highly non-linear optimisation problems such as the construction of financial trading strategies, where GAs are already employed (Salman et al. 2023).

# References


Cheng Xi Tsou. (2021) 'Genetic algorithm: 8 queens problem', *Medium*, 18 May. Available at: https://medium.com/nerd-for-tech/genetic-algorithm-8-queens-problem-b01730e673fd (Accessed: 11 November 2024).

Educative. (n.d.). *Solving the 8 queen problem using genetic algorithm*. Available at: https://www.educative.io/answers/solving-the-8-queen-problem-using-genetic-algorithm (Accessed: 11 November 2024)

Fang, Y. & Li, J. (2010) 'A review of tournament selection in genetic programming', *ISICA 2010 Conference Proceedings*, October. Available at: https://www.researchgate.net/publication/221494298 (Accessed: 11 November 2024).

Hasançebi, O. and Erbatur, F. (2000) 'Evaluation of crossover techniques in genetic algorithm-based optimum structural design', *Computers & Structures*, 78(3), pp. 435-448. Available at: https://doi.org/10.1016/S0045-7949(00)00089-4 (Accessed: 11 November 2024).

Greenstein, B.L., Elsey, D.C., and Hutchison, G.R. (2023) 'Determining best practices for using genetic algorithms in molecular discovery', *The Journal of Chemical Physics*, 159(9), 091501. Available at: https://doi.org/10.1063/5.0158053 (Accessed: 12 November 2024).

Jaworski, B., Kuczkowski, Ł., Śmierzchalski, R., and Kolendo, P. (2012) 'Extinction event concepts for the evolutionary algorithms', *Przegląd Elektrotechniczny (Electrical Review)*, 88(10b), pp. 252-255.

Cook, J.D. (2021) 'Queens on a donut', *The Endeavour*, 18 August. Available at: https://www.johndcook.com/blog/2021/08/18/queens-on-a-donut/ (Accessed: 12 November 2024).

Hammad, A.W. and Thannoon, B.N. (2007) 'Genetic algorithm versus particle swarm optimization in N-queen problem', *Journal of Al-Nahrain University*, 10(2), pp. 172-177.

Salman, O., Melissourgos, T., & Kampouridis, M., 2023. Optimization of trading strategies using a genetic algorithm under the directional changes paradigm with multiple thresholds. *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*. University of Essex.

Note: Generative AI – ChatGPT and Github Copilot – was used only as a tool in the development of the code seen in the Jupyter notebook. No complete solutions from Generative AI tools are presented in the submission.